

Politecnico di Milano

Anno accademico 2010-2011

Ingegneria del Software – Appello del 2 settembre 2011

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 1

Un'`OrganizzazioneBenStrutturata` (OBS) è costituita da un insieme di `Dipendenti` (D) e da un `ResponsabileOperativo` (RO), che a sua volta è un dipendente. I dipendenti hanno un salario: quello del RO non può superare il più alto degli altri dipendenti di più del 30%. Il salario dei dipendenti può oscillare del $+$ o $-$ 20% rispetto a un salario "standard".

Le classi `OrganizzazioneBenStrutturata` e `Dipendente` sono specificate come segue, ipotizzando che `Dipendente` deleghi a una classe`Persona` la gestione dei dati anagrafici: serve una `Persona` per creare un nuovo `Dipendente`. Inoltre, per semplicità, si consideri che il RO sia definito solo al momento dell'inizializzazione e che non si possano eliminare dipendenti dalla OBS.

```
public /*@ pure @*/ class Dipendente {
  //@ensures (*costruisce un nuovo dipendente, con salario salarioIniziale*)
  public Dipendente(Persona p, int salarioIniziale);

  //@ensures (*\result e' la Persona corrispondente a this *);
  public Persona persona();

  //@ensures (*\result e' il salario di this *);
  public int salario();
}

public class OrganizzazioneBenStrutturata {
  // definisce una nuova OBS con resp come RO e salario standard
  // pari a salarioStandard
  public OrganizzazioneBenStrutturata(ResponsabileOperativo resp,
                                      int salarioStandard)

  //@ensures (* \result e' true sse d e' parte di this, come RO o dipendente*)
  public /*@ pure @*/ boolean isDipendente (Dipendente d)

  //@ensures (* \result e' il responsabile di this*)
  public /*@ pure @*/  ResponsabileOperativo responsabile()

  //@ensures (*\result e' il salario standard per l'organizzazione*);
  public /*@ pure @*/ int salarioStandard()

 // aggiunge un dipendente con il salario assegnato
  public void addDipendente(Dipendente d, int salario)

  // aggiorna il salario del dipendente
  public void cambiaSalario(Dipendente d, int nuovoSalario)
}
```

Si chiede di:

1. Definire l'interfaccia della classe `ResponsabileOperativo` in modo opportuno. **Soluzione:**

```
public /*@ pure @*/ class ResponsabileOperativo extends Dipendente {
  //@ensures (*costruisce un nuovo ResponsabileOperativo, con salario salarioIniziale*)
  public ResponsabileOperativo(Persona p, int salarioIniziale);
}
```

2. Esprimere in JML i vincoli di cui sopra per la classe `OrganizzazioneBenStrutturata` come invariante astratto.

   Soluzione: Definiamo per brevita' un predicato:

```
//@ensures \result <==> d.salario()>= salStand*0.8 && d.salario()<= salStand*1.2
//@ && r.salario()>=d.salario()*1.3;
public boolean salarioOk(int salStand, ResponsabileOperativo r, salario s)
```

   L'invariante pubblico, che considera oltre ai vincoli sui salari anche il vincolo (non esplicito, ma ovvio dal contesto) che non vi siano due dipendenti che sono la stessa persona, e':

```
//@public invariant (\forall Dipendente d; isDipendente(d) && d!=responsabile();
//@  salarioOk(salarioStandard(), responsabile(), d)
 && !(\exists Dipendente d1; isDipendente(d1); d!= d1 && d.persona() == d1.persona()));
```

3. Specificare in JML i metodi `OrganizzazioneBenStrutturata`, `addDipendente` e `cambiaSalario`. Le specifiche devono essere definite come operazioni parziali, aggiungendo eventuali vincoli mediante opportuna precondizione, allo scopo di verificare l'invariante astratto.

   Soluzione:

```
//@requires resp != null && salarioStandard >0;
//@ensures this.responsabile() == resp && this.salarioStandard() == salarioStandard &&
//@ (\forall Dipendente d; isDipendente(d); d==resp); //non ci sono altri dipendenti
 public OrganizzazioneBenStrutturata(ResponsabileOperativo resp,
                                     int salarioStandard)

//@requires d!=null && salario>0 && salarioOk(salarioStandard(), responsabile(), d)
//@ && !isDipendente(d)
//@ && !(\exists Dipendente d1; isDipendente(d1); d!= d1&&d.persona()==d1.persona());
//@ensures isDipendente(d) && (\forall Dipendente d1; d1!= d;
//@ \old(isDipendente(d1)) <==> isDipendente(d1);
  public void addDipendente(Dipendente d, int salario)

Per la cambiaSalario occorre prestare attenzione al fatto che Dipendente e' immutabile:

//@requires d!=null && nuovoSalario>0 && salarioOk(salarioStandard(), responsabile(), d)
//@ensures !isDipendente(d)
//@  && (\exists Dipendente d1; isDipendente(d1); d1.persona()==d.persona() &&
//@     d1.salario == nuovoSalario);
  public void cambiaSalario(Dipendente d, int nuovoSalario)
```

4. Specificare in JML `cambiaSalario` come operazione totale, in modo che l'invariante sia verificato.

   Soluzione: Le requires vanno trasformate in eccezioni...

5. Si consideri una classe definita come segue:

```
public class DipendenteSpeciale {
  //@ensures (*costruisce un DipendenteSpeciale, con salario salarioIniziale*)
  public DipendenteSpeciale(Persona p, int salarioIniziale);
  ...qui gli stessi metodi salario() e persona() di Dipendente...
  //@ensures salario()==salario;
    public void nuovoSalario(int salario)
}
```

   E' corretto definire la classe DipendenteSpeciale come erede di Dipendente? Cosa succederebbe all'invariante astratto di OBS se nella definizione di OBS si usasse `DipendenteSpeciale` invece di `Dipendente`? Giustificare le risposte.

Soluzione: No, la classe DipendenteSpeciale e' mutabile, e le eventuali mutazioni possono essere osservate anche con il metodo salario() gia' definito in Dipendente: e' quindi violata la regola delle proprieta'. Se nonostante questo si procedesse con la definizione di DipendenteSpeciale come erede di Dipendente (cosa comunque possibile in Java), ad esempio l'invariante pubblico della classe OrganizzazioneBenStrutturata potrebbe essere violato, aggiungendo un DipendenteSpeciale e poi modificandone il salario con nuovoSalario.

6. Si consideri il seguente rep per la classe OBS.

```
private RO ro;
private ArrayList<Dipendente> dipendenti;
private int salarioStandard;
```

Scrivere un invariante di rappresentazione adeguato e, in base alla conoscenza dell'invariante e del rep, implementare il metodo `cambiaSalario`.

Soluzione: Oltre a condizioni consuete sul rep, occorre stabilire se ro e' in dipendenti oppure no. E' piu' naturale che non vi sia, essendo gia' memorizzato a parte nel rep. Si noti che le condizioni su salarioOk sono parte dell'inv. astratto e non sono da ripetere nell'inv. di rappresentazione; cosi' pure le relazioni fra il rep e i metodi osservatori (che costituiscono invece la funzione di astrazione).

```
//@ private invariant ro != null &&
//@ (\forall int i; 0\le i && i<dipendenti().size(); dipendenti != null) &&
//@ !dipendenti().contains(ro);
```

L'implementazione (per semplicita', nel caso con le precondizioni) deve considerare che Dipendente e' classe immutabile, e che il responsabile Operativo e' un dipendente come gli altri (ma non e' memorizzato nell'ArrayList dipendenti):

```
public void cambiaSalario(Dipendente d, int nuovoSalario) {
 if (ro.equals(d)) {
     RO r = new ResponsabileOperativo(d.persona(), nuovoSalario);
     ro = r;
 }
 else {
     Dipendente d1 = new Dipendente(d.persona(), nuovoSalario);
     dipendenti.remove(d);
     dipendenti.add(d1);
 }
}
```

Esercizio 2

Definire, motivando la risposta, cosa stampa il seguente frammento di codice Java:

```
public class ClassA {
   private int a;

   public int value() {return a;}

   public void stampa(ClassA p) {
      System.out.println(this.value()+p.value());
   }
}

public class ClassB extends ClassA {
   public void stampa(ClassB p) {
      System.out.println(this.value()+p.value()+1);
   }

   public void stampa(ClassA p) {
      System.out.println(this.value()+p.value()+2);
   }
}

public class ClassC extends ClassB {
   public void stampa(ClassC p) {
  System.out.println(this.value()+p.value()+3);
   }

   public void stampa(ClassA p) {
      System.out.println(this.value()+p.value()+4);
   }
}

public class Test {
   public static void main(String[] args) {
      ClassA v1 = new ClassA();
      ClassA v2 = new ClassB();
      ClassA v3 = new ClassC();
      ClassB v4 = new ClassB();
      ClassC v5 = new ClassC();

      v1.stampa(v1);
      v1.stampa(v2);
      v3.stampa(v2);
      v4.stampa(v3);
      v5.stampa(v1);
      v5.stampa(v4);
      v4.stampa(v5);
   }
}
```

Soluzione: 0 0 4 2 4 1 1

Esercizio 3

Solitamente, quando si vogliono prelevare dei soldi da uno sportello bancomat, si compiono le seguenti operazioni:

- Il ciente inserisce la tessera nel lettore
- Il cliente digita il codice segreto
- Il sistema controlla che il codice sia corretto (e supponiamo sia corretto)
- Il sistema propone le diverse quantità prelevabili (6 diverse alternative)
- Il cliente sceglie l'ammontare desiderato
- Se richiesto, il sistema stampa la ricevuta
- Il sistema restituisce la tessera, da' i soldi e toglie la somma dal saldo del conto del cliente
- Il sistema termina l'operazione e si prepara per un'altra operazione

Identificare gli attori/oggetti coinvolti, utilizzando una granularità opportuna, attraverso un *diagramma delle classi* e rappresentare lo scenario con un *diagramma di sequenza*.